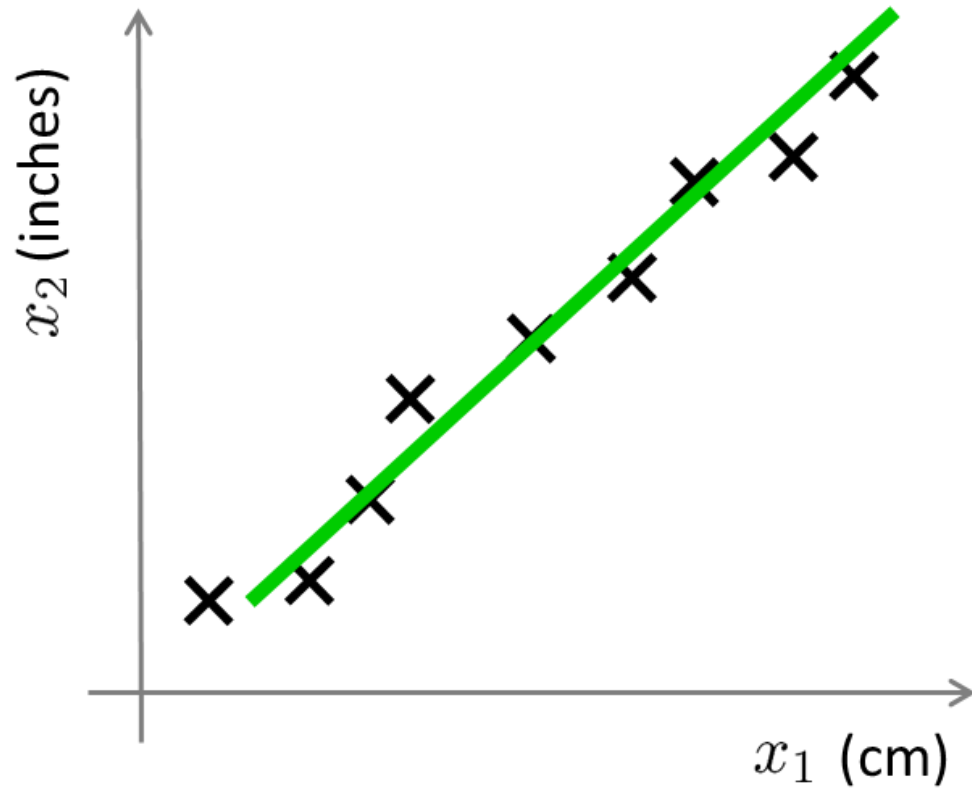


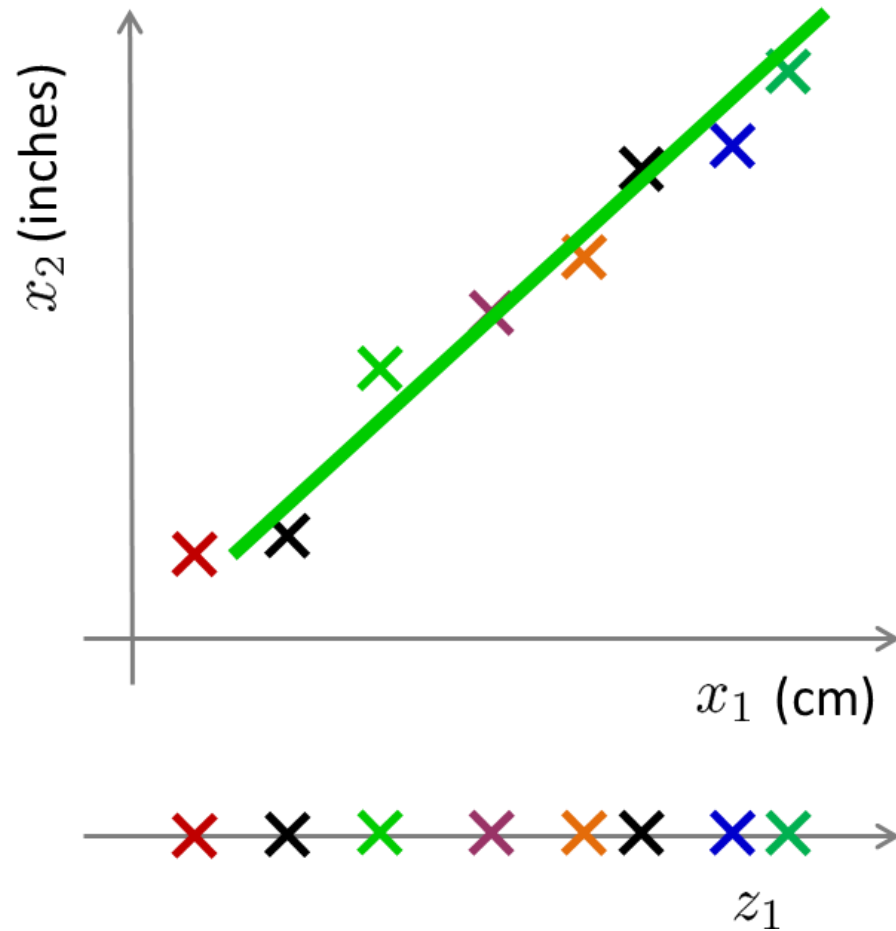
Dimensionality Reduction

Prof. Giuseppe Serra

Università degli Studi di Udine



Reduce data from
2D to 1D



Reduce data from
2D to 1D

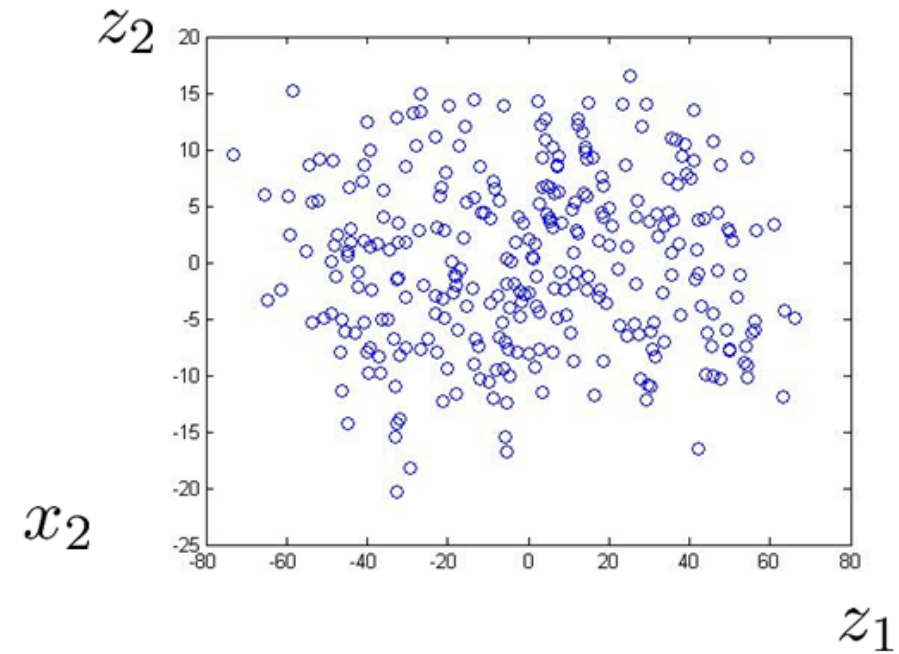
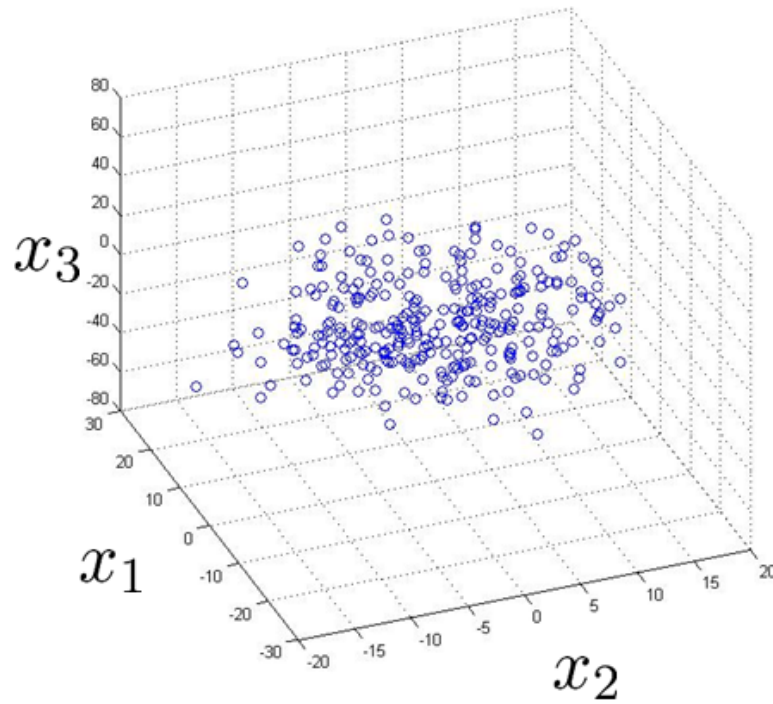
$$x^{(1)} \rightarrow z^{(1)}$$

$$x^{(2)} \rightarrow z^{(2)}$$

$\vdots$

$$x^{(m)} \rightarrow z^{(m)}$$

Reduce data from 3D to 2D



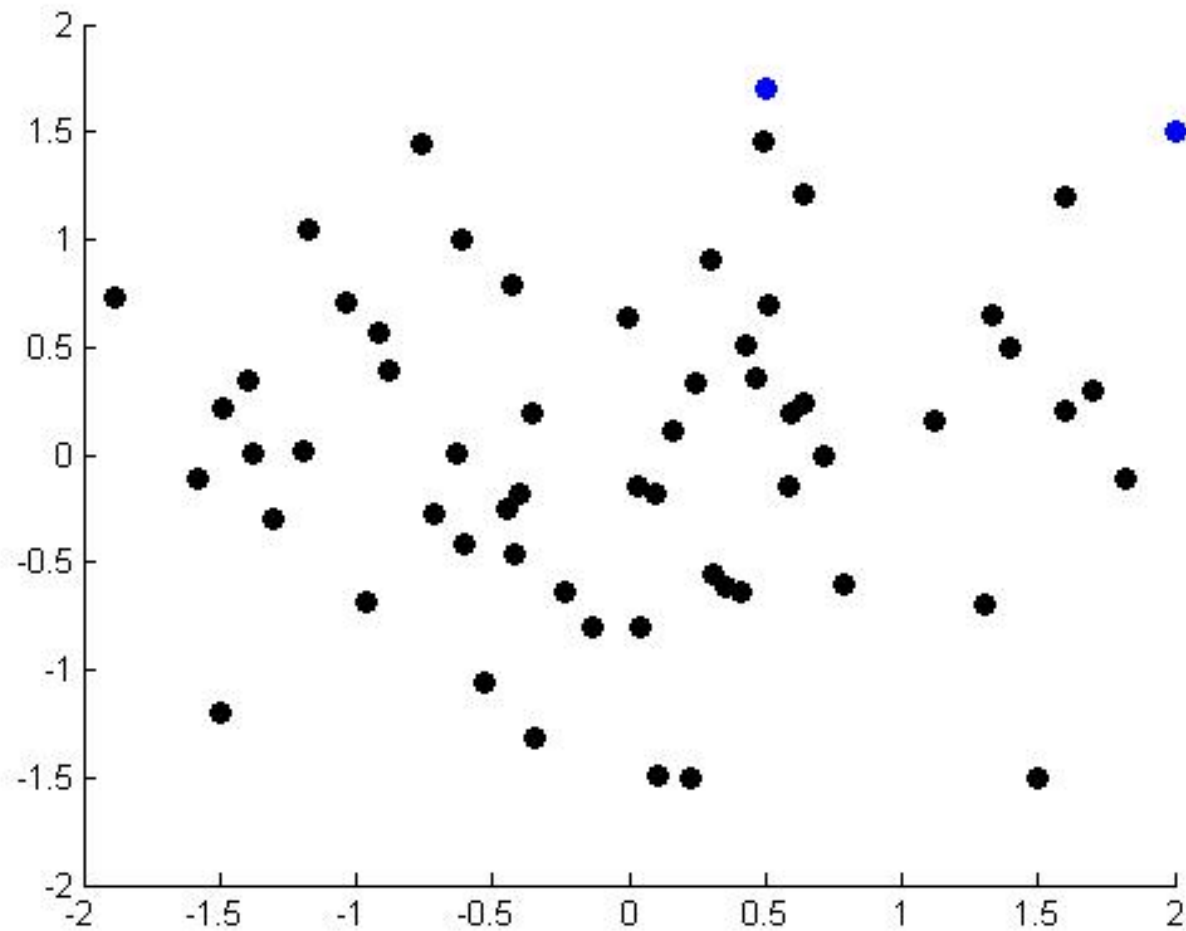
Data Visualization

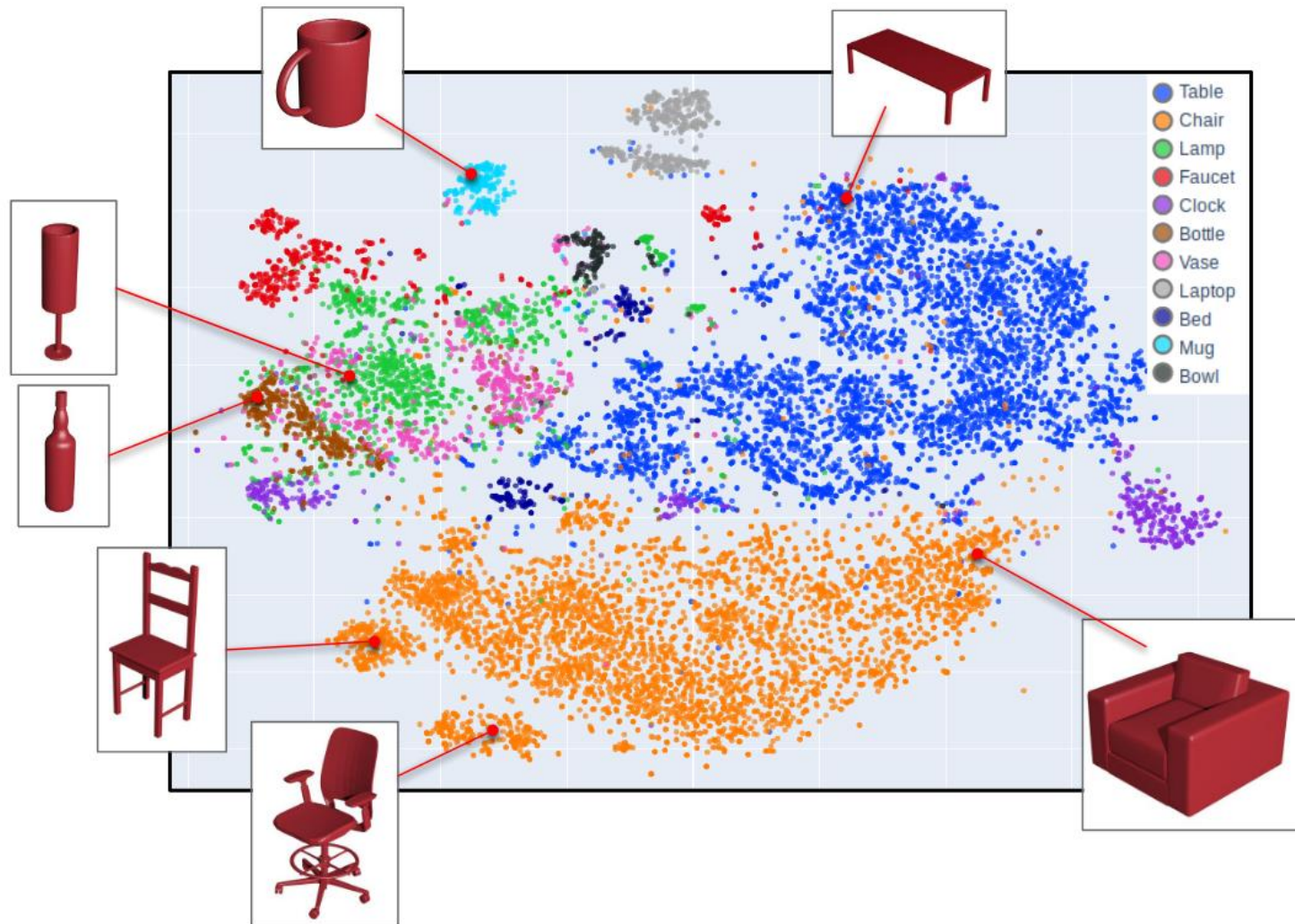
Country	GDP (trillions of US\$)	Per capita GDP (thousands of intl. \$)	Human Development Index	Life expectancy	Poverty Index (Gini as percentage)	Mean household income (thousands of US\$)	...
Canada	1.577	39.17	0.908	80.7	32.6	67.293	...
China	5.878	7.54	0.687	73	46.9	10.22	...
India	1.632	3.41	0.547	64.7	36.8	0.735	...
Russia	1.48	19.84	0.755	65.5	39.9	0.72	...
Singapore	0.223	56.69	0.866	80	42.5	67.1	...
USA	14.527	46.86	0.91	78.3	40.8	84.3	...
...	...	...	...	...	...	...	...

[resources from en.wikipedia.org]

Data Visualization

Country	z_1	z_2
Canada	1.6	1.2
China	1.7	0.3
India	1.6	0.2
Russia	1.4	0.5
Singapore	0.5	1.7
USA	2	1.5
...	...	...

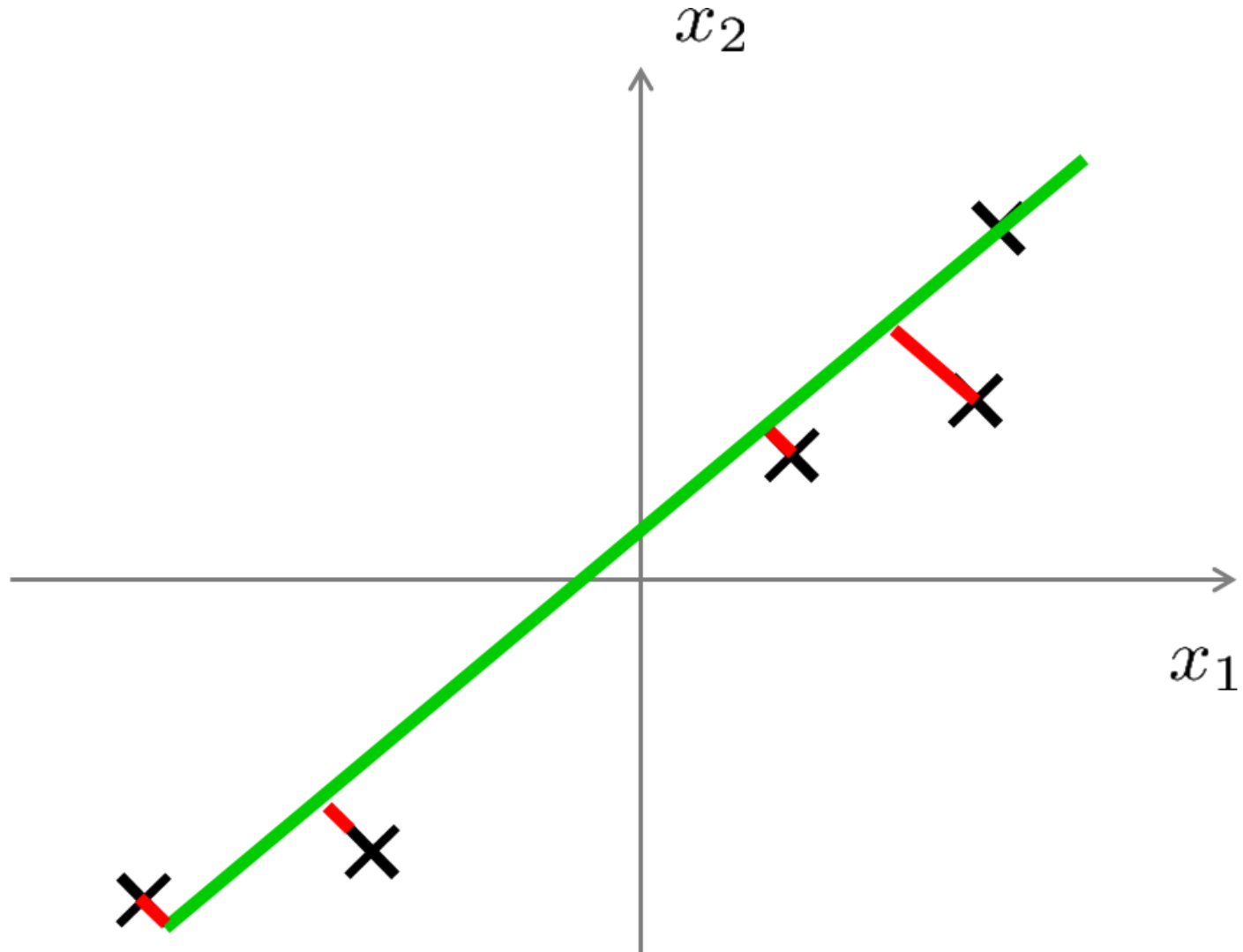




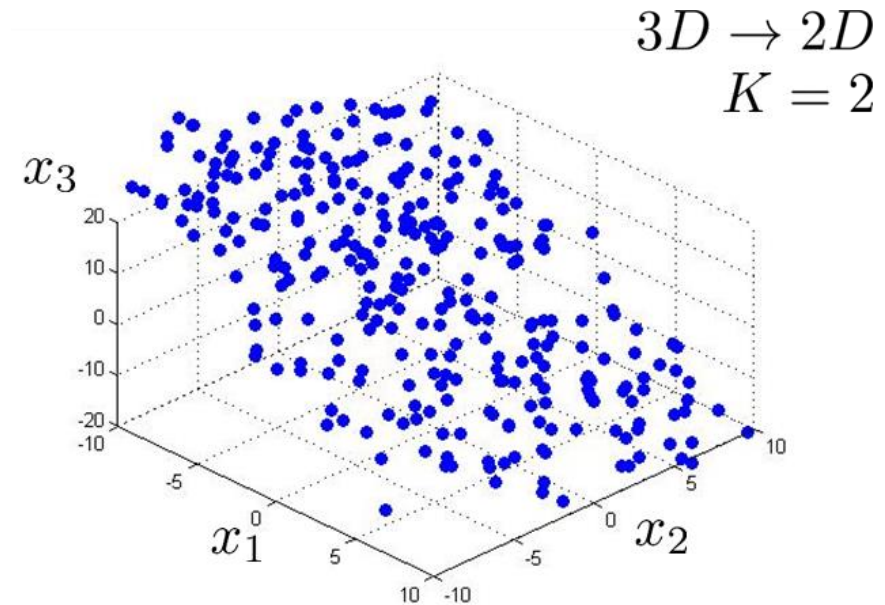
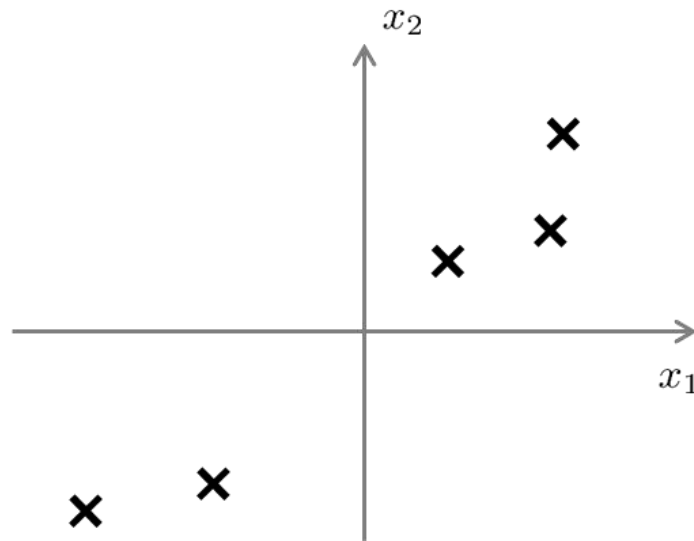
Principal Component Analysis

Problem Formulation

Principal Component Analysis (PCA)



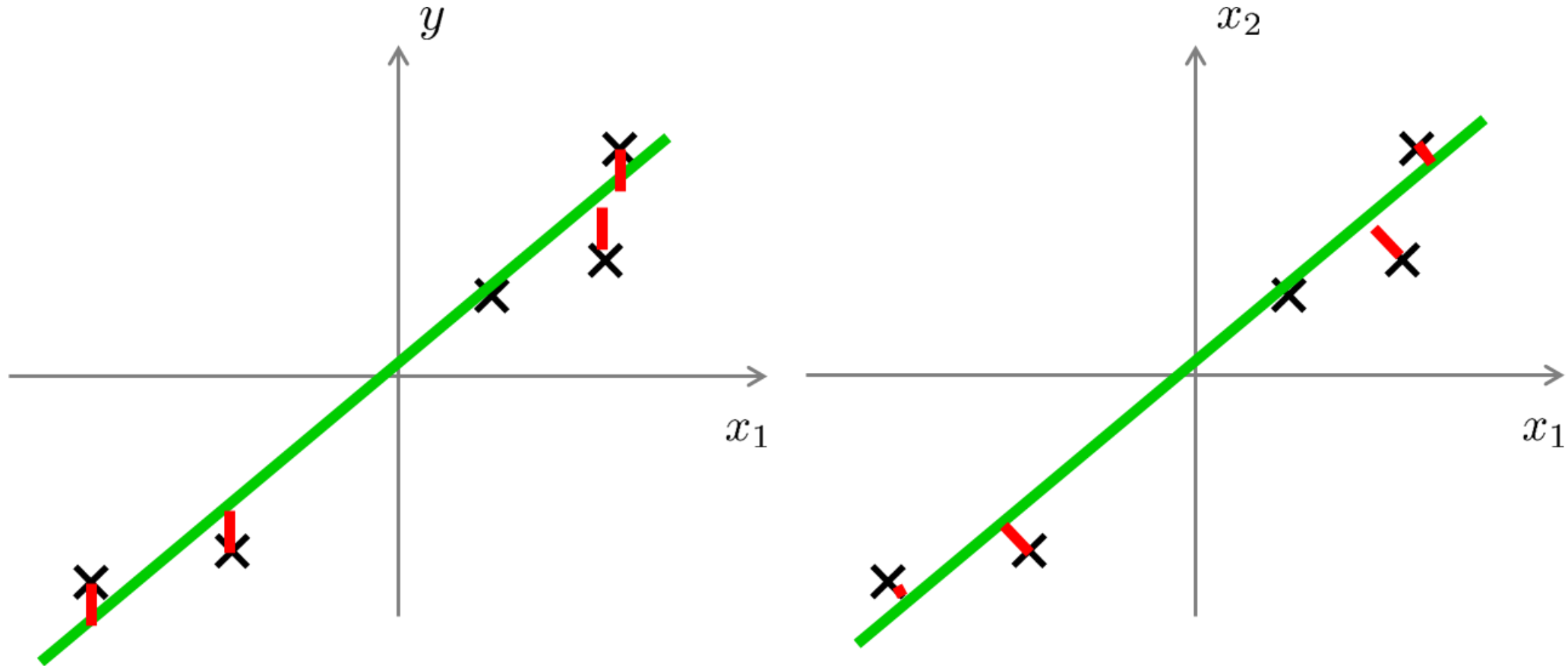
Principal Component Analysis (PCA)



Reduce from 2-dimension to 1-dimension: Find a direction (**a vector** $u^{(1)} \in \mathbb{R}^n$) onto which to project the data so as to minimize the projection error.

Reduce from n -dimension to k -dimension: Find k vectors $u^{(1)}, u^{(2)}, \dots, u^{(k)}$ onto which to project the data, so as to minimize the projection error.

PCA is not linear Regression



Principal Component Analysis Algorithm

Data Preprocessing

Training set: $x^{(1)}, x^{(2)}, \dots, x^{(m)}$

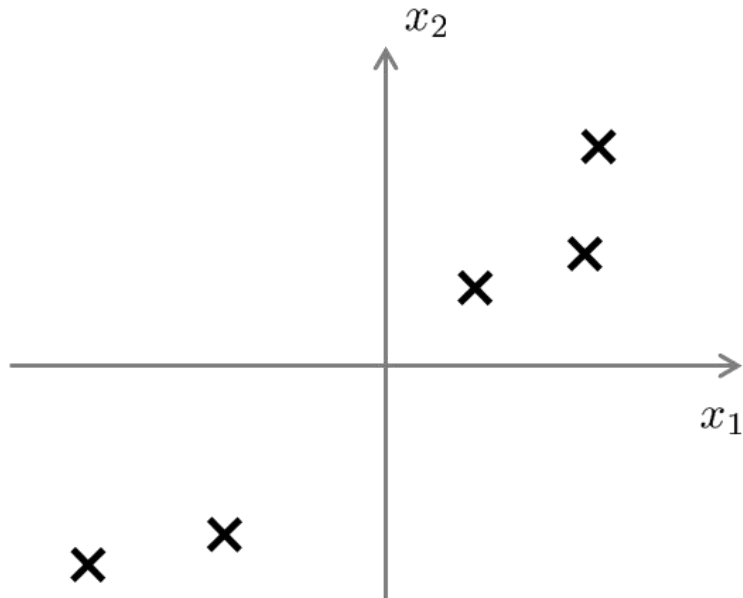
Preprocessing (feature scaling/mean normalization):

$$\mu_j = \frac{1}{m} \sum_{i=1}^m x_j^{(i)}$$

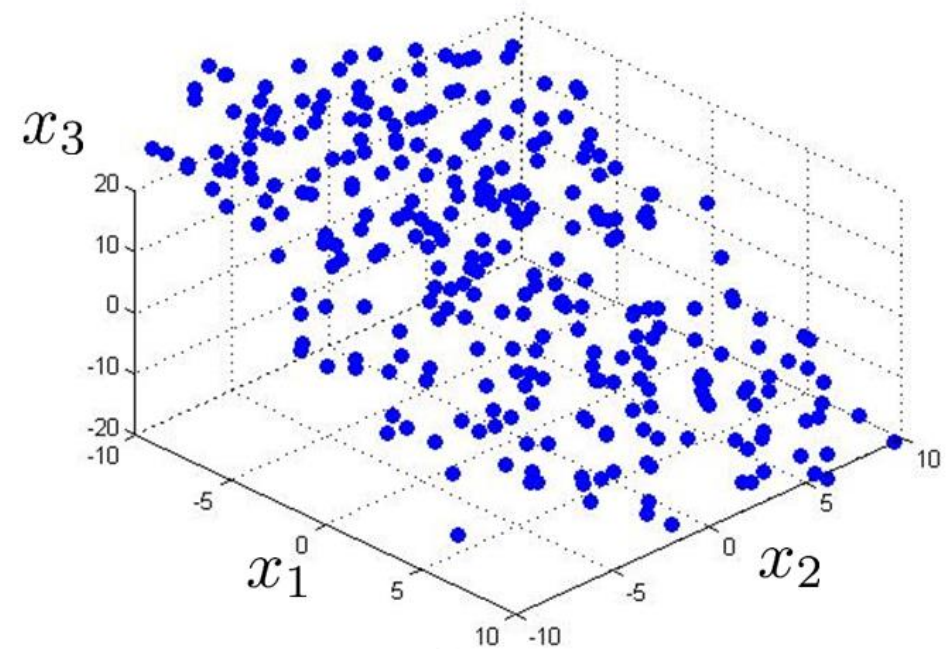
Replace each $x_j^{(i)}$ with $x_j - \mu_j$.

If different features on different scales (e.g., x_1 = size of house, x_2 = number of bedrooms), scale features to have comparable range of values.

PCA Algorithm



Reduce data from 2D to 1D



Reduce data from 3D to 2D

PCA Algorithm

Reduce data from n -dimensions to k -dimensions

Compute “covariance matrix”:

$$\Sigma = \frac{1}{m} \sum_{i=1}^n (x^{(i)})(x^{(i)})^T$$

Compute “eigenvectors” of matrix Σ :

$$[U, S, V] = \text{svd}(\text{Sigma}) ;$$

svd (singular value decomposition)

PCA Algorithm

From `[U,S,V] = svd(Sigma)` , we get:

$$U = \begin{bmatrix} | & | & \dots & | \\ u^{(1)} & u^{(2)} & & u^{(n)} \\ | & | & & | \end{bmatrix} \in \mathbb{R}^{n \times n}$$

PCA Algorithm summary

After mean normalization (ensure every feature has zero mean) and optionally feature scaling:

$$\mathbf{Sigma} = \frac{1}{m} \sum_{i=1}^m (x^{(i)})(x^{(i)})^T$$

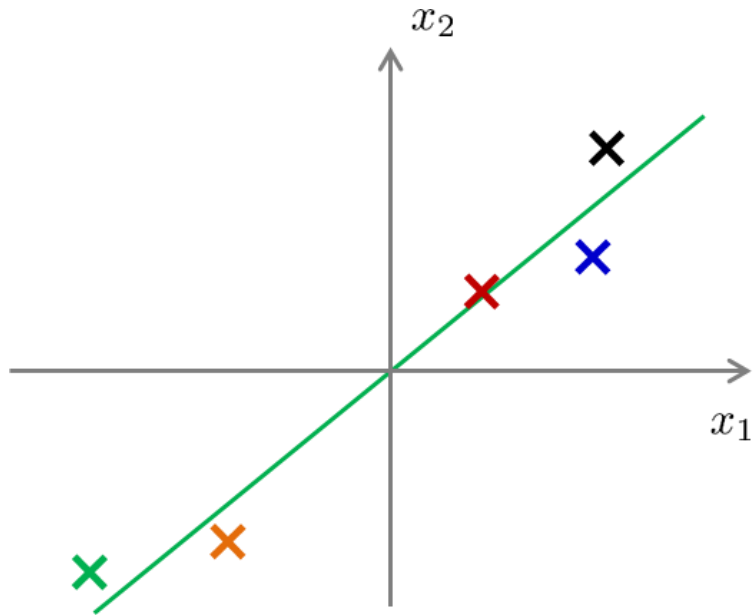
```
[U,S,V] = svd(Sigma);
```

```
Ureduce = U(:,1:k);
```

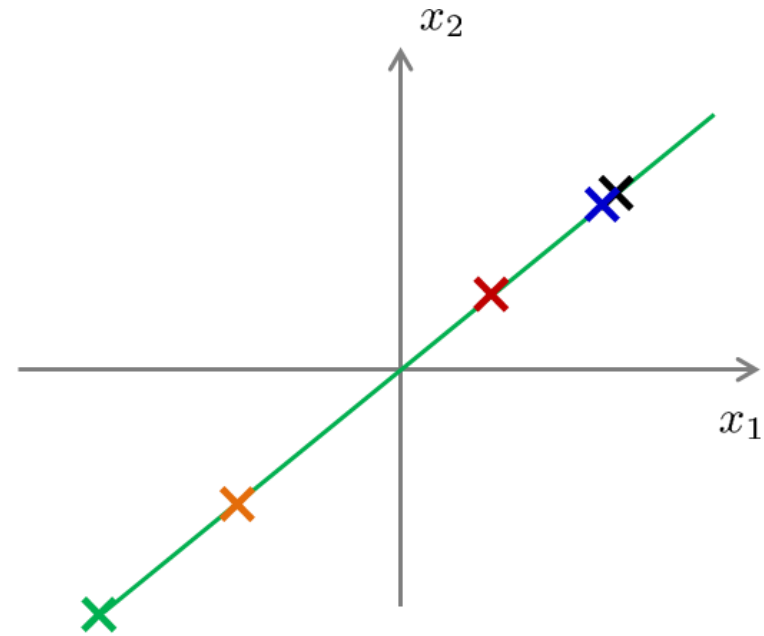
```
z = Ureduce'*x;
```

Reconstruction from compressed representation

PCA Algorithm



$$z = U_{reduce}^T x$$



$$X_{approx} = U_{reduce} z$$

Choosing the number of principal components

Choosing k (number of principal components)

Average squared projection error:

Total variation in the data:

Typically, choose k to be smallest value so that

$$\frac{\frac{1}{m} \sum_{i=1}^m \|x^{(i)} - x_{approx}^{(i)}\|^2}{\frac{1}{m} \sum_{i=1}^m \|x^{(i)}\|^2} \leq 0.01 \quad (1\%)$$

“99% of variance is retained”

Choosing k (number of principal components)

Algorithm:

Try PCA with $k = 1$

Compute $U_{reduce}, z^{(1)}, z^{(2)}, \dots, z^{(m)}, x_{approx}^{(1)}, \dots, x_{approx}^{(m)}$

Check if

$$\frac{\frac{1}{m} \sum_{i=1}^m \|x^{(i)} - x_{approx}^{(i)}\|^2}{\frac{1}{m} \sum_{i=1}^m \|x^{(i)}\|^2} \leq 0.01?$$

$$[U, S, V] = \text{svd}(\text{Sigma})$$

$$1 - \frac{\sum_{i=1}^k S_{ii}}{\sum_{i=1}^N S_{ii}}$$

Choosing k (number of principal components)

$[U, S, V] = \text{svd}(\text{Sigma})$

Pick smallest value of k for which

$$\frac{\sum_{i=1}^k S_{ii}}{\sum_{i=1}^m S_{ii}} \geq 0.99$$

(99% of variance retained)

Advice for applying PCA

Supervised Learning Speedup

$$(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})$$

Extract inputs:

$$\text{Unlabeled dataset: } x^{(1)}, x^{(2)}, \dots, x^{(m)} \in \mathbb{R}^{10000}$$

$$\downarrow \text{PCA}$$

$$z^{(1)}, z^{(2)}, \dots, z^{(m)} \in \mathbb{R}^{1000}$$

New training set:

$$(z^{(1)}, y^{(1)}), (z^{(2)}, y^{(2)}), \dots, (z^{(m)}, y^{(m)})$$

Note: Mapping $x^{(i)} \rightarrow z^{(i)}$ should be defined by running PCA only on the training set. This mapping can be applied as well to the examples $x_{cv}^{(i)}$ and $x_{test}^{(i)}$ in the cross validation and test sets.

- Compression
 - Reduce memory/disk needed to store data
 - Speed up learning algorithm
- Visualization ($k=2$ or $k=3$)

PCA is sometimes used where it shouldn't be

Design of ML system:

- Get training set $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})\}$
- Run PCA to reduce $x^{(i)}$ in dimension to get $z^{(i)}$
- Train logistic regression on $\{(z^{(1)}, y^{(1)}), \dots, (z^{(m)}, y^{(m)})\}$
- Test on test set: Map $x_{test}^{(i)}$ to $z_{test}^{(i)}$. Run $h_{\theta}(z)$ on $\{(z_{test}^{(1)}, y_{test}^{(1)}), \dots, (z_{test}^{(m)}, y_{test}^{(m)})\}$

How about doing the whole thing without using PCA?

Before implementing PCA, first try running whatever you want to do with the original/raw data $x^{(i)}$. Only if that doesn't do what you want, then implement PCA and consider using $z^{(i)}$.

Dimensionality Reduction using Neural Networks

Encoder - Decoder

